

***GReen And SuStainable – kNowledge
EXpanded freight Transport in cities***

**INTERACTIVE WEB-BASED SERVICE FOR
CITY LOGISTICS DATA ANALYSIS - WEB
APPLICATION (USER DOCUMENTATION)**

Table of contents:

1	User documentation	3
1.1	Application description	3
1.2	Navigation bar	3
1.2.1	Menu icon	4
1.2.2	Logos	4
1.2.3	“Home” and “About” buttons	4
1.2.4	Additional information icon	4
1.3	Measure parameters form	5
1.3.1	“Country”, “State” and “City” fields	6
1.3.2	“Detector” field.....	6
1.3.3	“Date” field	7
1.3.4	“Time” field	8
1.3.5	“Vehicle Types” field.....	8
1.3.6	“Pollution Types” field.....	8
1.3.7	“Weather Stability” field	9
1.3.8	“Area” slider	10
1.3.9	“Calculate” button.....	10
1.3.10	“Clear” button	11
1.3.11	Loading screen.....	11
1.3.12	Additional information.....	11
1.4	Map.....	12
1.5	Footer.....	12
1.6	“About” subsite	13
2	Technical documentation.....	14
2.1	Documentation purpose	14
2.2	Design and implementation constraints	14
2.3	Functional requirements	14
2.3.1	List of functional requirements.....	14
2.3.2	Detailed description of functional requirements	14
2.4	Non-functional requirements	23
2.5	System architecture.....	24
2.5.1	Technologies used	25
2.5.2	Communication between frontend and backend.....	26
2.5.3	Mathematical model	26
2.5.4	Database design	27
2.6	Test scenarios.....	28

1 USER DOCUMENTATION

The user documentation contains all the necessary information about the usage of the website that is hosted under the domain www.green-tronic.eu. The project is also available in form of [GitHub repository](#). It is intended for users who wish to independently analyze air pollution dispersion based on real topographical and traffic data collected within the GRASS-NEXT project

1.1 Application description

The GRASS-NEXT web application is an interactive platform designed to allow users to self-analyze air pollution dispersion using historical traffic data. The system processes information obtained from TOPO detectors, which are installed near selected roads and automatically record the number of passing vehicles, their classification (vehicle group), and their velocity. These measurements are periodically transmitted to the server where they are validated, aggregated, and stored together with location metadata and weather conditions.

The following part of the documentation will describe all the subpages and elements of the user interface along with their purpose.

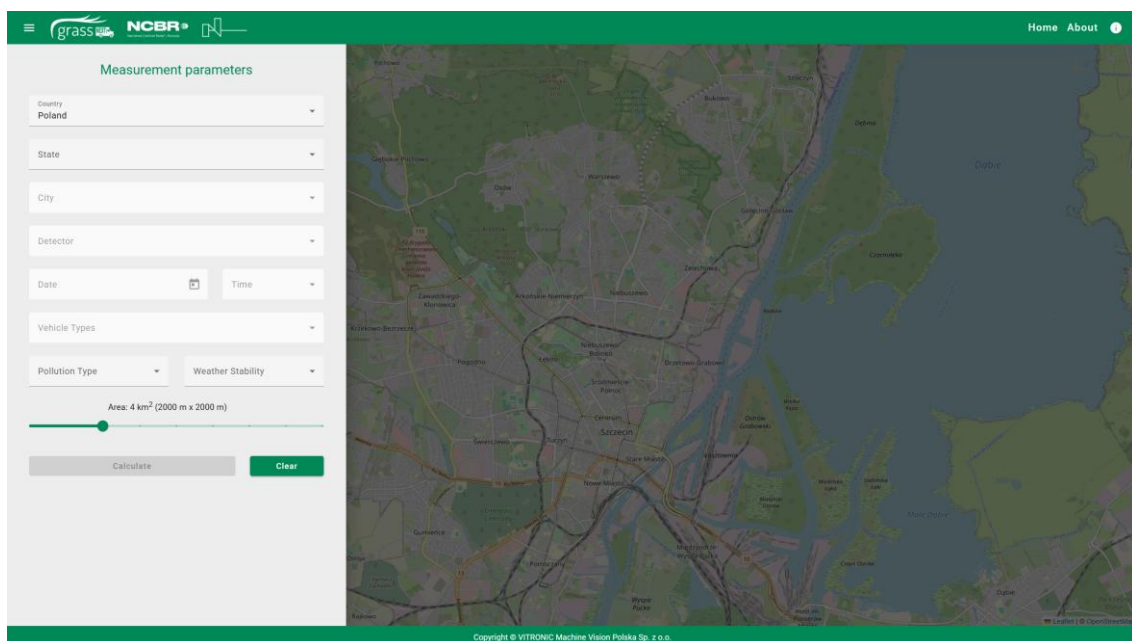


Figure 1.1. The appearance of the green-tronic.eu website.

1.2 Navigation bar

In the upper part of the screen there is a navigation bar. It's split in two parts. On the left there is an menu icon, as well as logos of affiliated partners and on the right – “Home” and “About” buttons alongside legend icon. All aforementioned elements will be described in detail in the latter part of the documentation.

1.2.1 Menu icon

The menu icon allows user to either show or hide left panel with form.



Figure 1.2. Menu icon.

1.2.2 Logos

On the left side of the navigation bar logos of affiliated partners are placed. Each of them is connected to the main website of an appropriate company via link.



Figure 1.3. Navigation bar logos.

1.2.3 “Home” and “About” buttons

Both “Home” and “About” buttons takes user to the appropriate subpage.

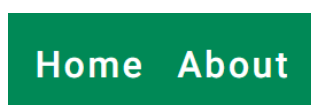


Figure 1.4. „Home” and „About” buttons.

1.2.4 Additional information icon

Additional information icon helps user to understand the functionality of each part of the website by displaying information about the most important elements of the main view.



Figure 1.5. Additional information icon.

After opening the legend by clicking the icon shown above users can get information about content of each form field alongside its purpose and data provided.

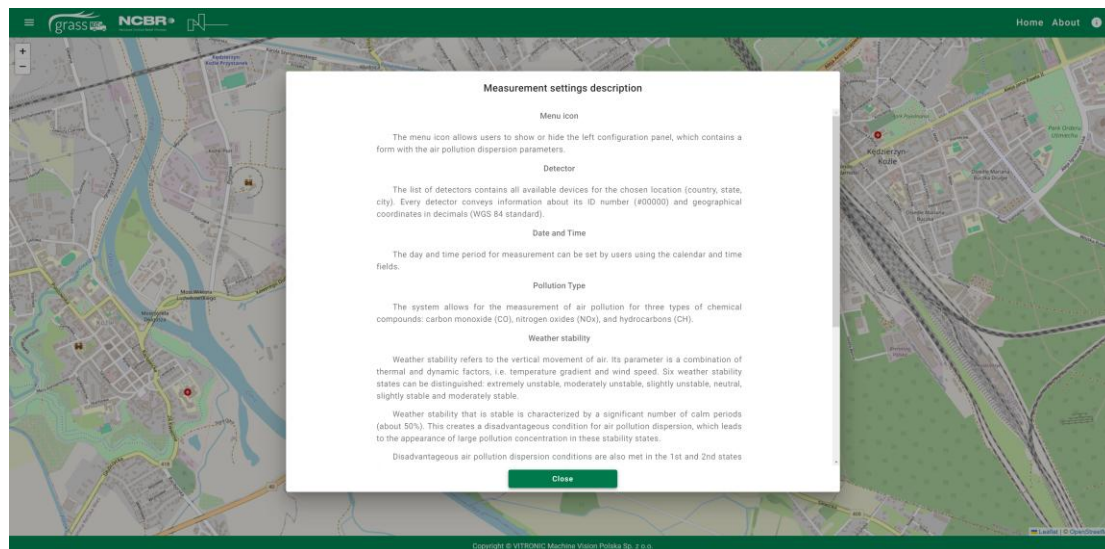


Figure 1.6. Legend.

1.3 Measure parameters form

The most important part of the application's main view is the measurement parameters form. Through its interface data gathered from detectors are shown. They can be later used to conduct measurements of air pollution dispersion. After choosing the necessary parameters, application's computing service carries out calculations that are later displayed on map in the form of isolines.

Figure 1.7. Measure parameters form.

1.3.1 “Country”, “State” and “City” fields

The selection of the location in which available detectors will be searched out is carried out by filling “Country”, “State” and “City” fields. After appropriate values are chosen, TOPO detector list will be unlocked. Locations are received from the database.

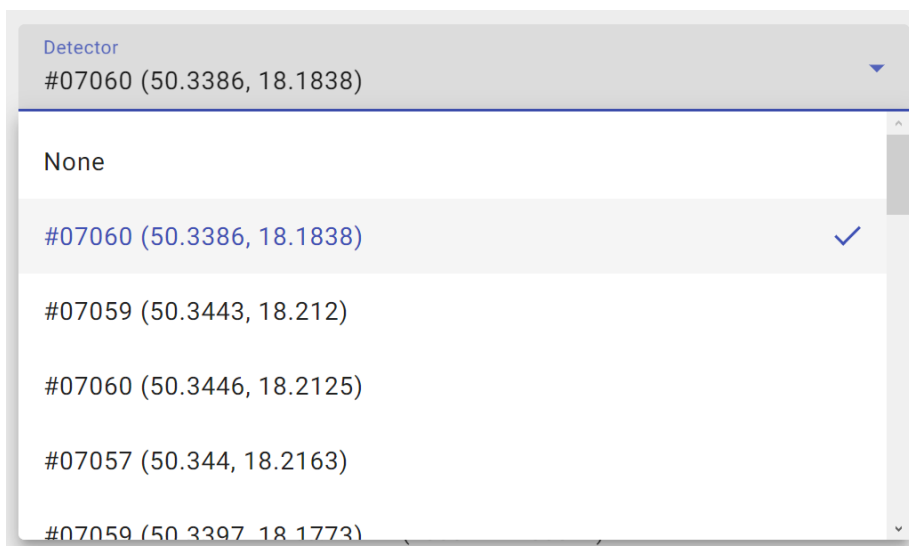


A screenshot of a web form with three stacked dropdown menus. The first menu is labeled 'Country' and has 'Poland' selected. The second menu is labeled 'State' and has 'Opole Voivodeship' selected. The third menu is labeled 'City' and has 'Kędzierzyn-Koźle' selected. Each menu has a downward arrow on the right side.

Figure 1.8. Example of filled location fields.

1.3.2 “Detector” field

After the location is selected, user will be allowed to choose a detector, which has collected data in a specified place. Sensors’ names will be presented as a six-digit identifier preceded by # sign (eg. #000000), as well as precise location in the form of geographic coordinates placed in brackets. They are presented in generally accepted WGS 84 standard.



A screenshot of a dropdown menu for selecting a detector. The menu is titled 'Detector' and shows a list of options. The first option is '#07060 (50.3386, 18.1838)' which is highlighted in grey and has a blue checkmark to its right. Below it are several other options: 'None', '#07059 (50.3443, 18.212)', '#07060 (50.3446, 18.2125)', '#07057 (50.344, 18.2163)', and '#07059 (50.3397, 18.1773)'. The list is scrollable, as indicated by a scrollbar on the right.

Figure 1.9. Dropdown TOPO detectors list.

When the sensor is chosen, the appropriate location will be placed on the map in the form of a red maker. Additionally, a blue line representing road will be drawn along with it’s start and end defined by blue pins.

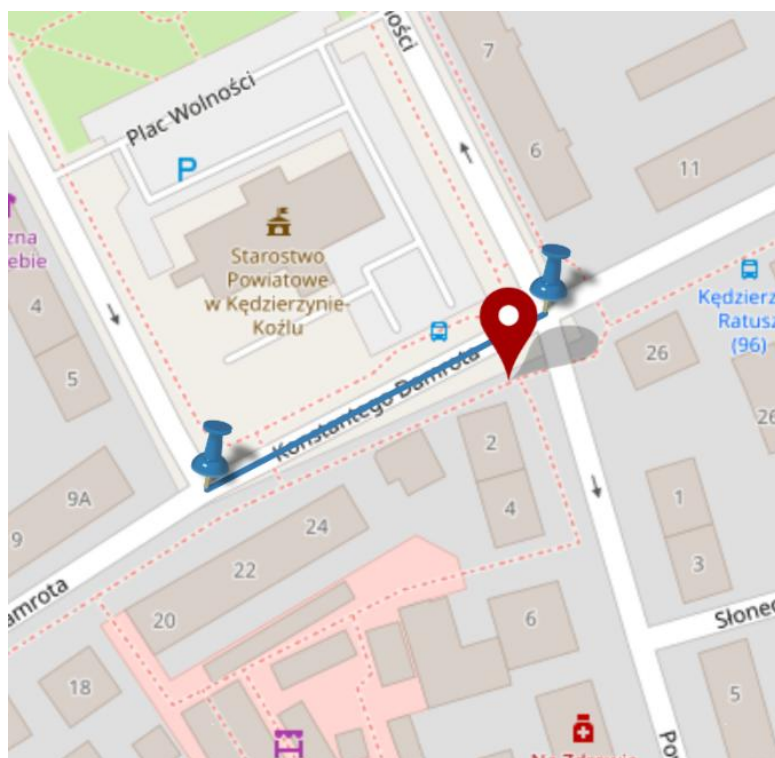


Figure 1.10. Detector location and road marked on map.

1.3.3 “Date” field

In the calendar, which is available after clicking the “Date” field, dates in which detector was collecting measurement data will be marked with green circles.

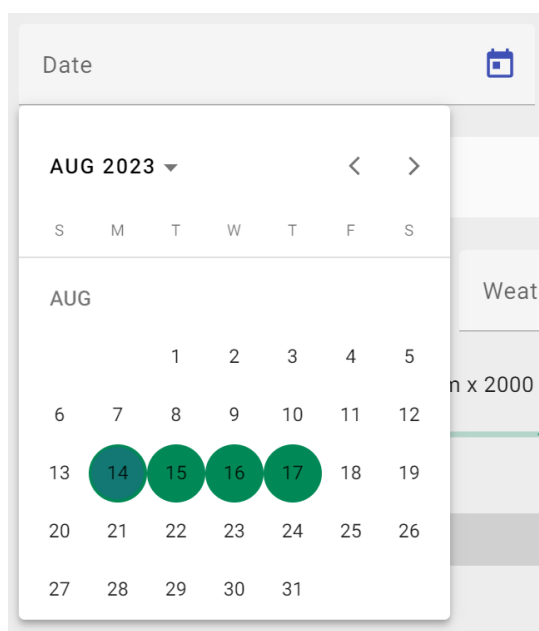
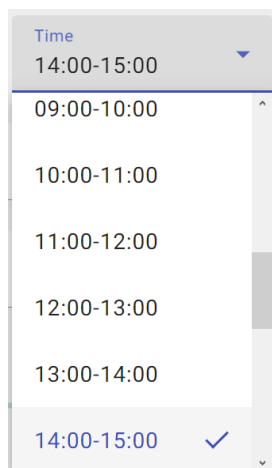


Figure 1.11. The dates on which the detector collected data.

1.3.4 “Time” field

For the chosen date it’s necessary to specify an hour of traffic measurement. Dates are split into one-hour intervals.

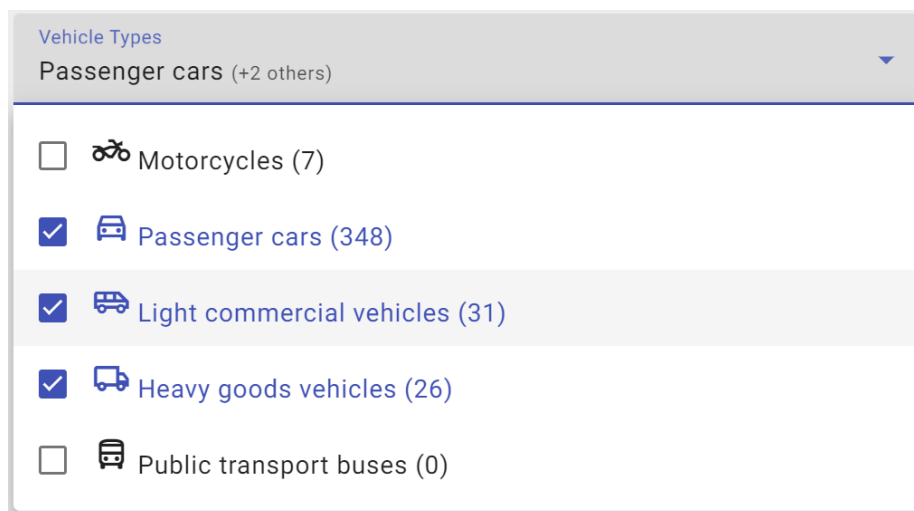


The screenshot shows a dropdown menu titled "Time". The currently selected option is "14:00-15:00", which is highlighted in blue and has a blue checkmark to its right. Other visible options in the list include "09:00-10:00", "10:00-11:00", "11:00-12:00", "12:00-13:00", and "13:00-14:00".

Figure 1.12. Selection of traffic measurement time.

1.3.5 “Vehicle Types” field

After date and time are selected, user can mark vehicle types, for which he want the air pollution dispersion measurement to be carried out. In app there are 5 types of vehicles defined: motorcycles, passenger cars, light commercial vehicles, heavy goods vehicles and public transport buses.



The screenshot shows a screen titled "Vehicle Types". At the top, there is a header bar with the text "Passenger cars (+2 others)" and a dropdown arrow. Below this, there is a list of vehicle types with checkboxes and counts:

- ☐  Motorcycles (7)
- ☒  Passenger cars (348)
- ☒  Light commercial vehicles (31)
- ☒  Heavy goods vehicles (26)
- ☐  Public transport buses (0)

Figure 1.13. Selection of chosen vehicle types.

1.3.6 “Pollution Types” field

In the app it’s possible to choose one of three kinds of air pollution. These are carbon monoxide (CO), nitrogen oxides (NO_x) and hydrocarbons (CH).

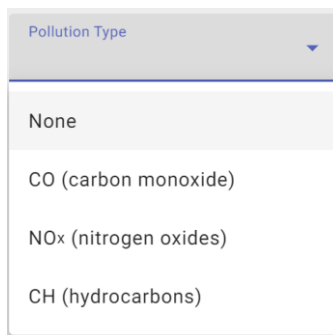


Figure 1.14. Air pollution selection field.

Each of the compounds is marked on map with a different color. In order it's color orange, navy blue and maroon.

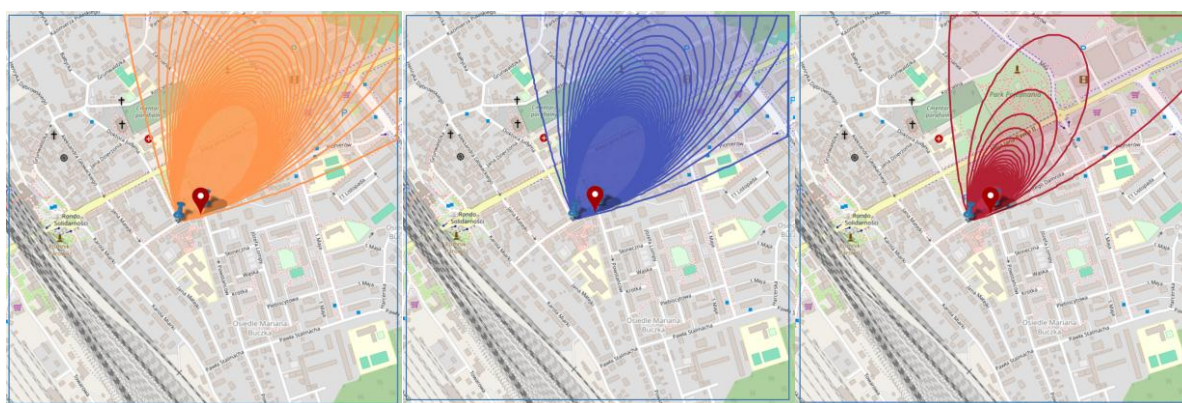


Figure 1.15. Different pollution types for the same location.

1.3.7 “Weather Stability” field

User can choose weather stability, for which the air pollution dispersion will be measured. This parameter influences the shape of isolines presented on the map. More information about each weather stability can be found in the applications’ legend.

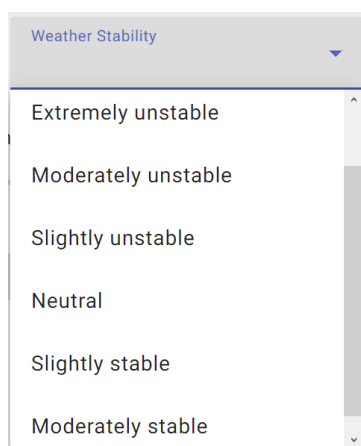


Figure 1.16. Weather stability selection field.

1.3.8 “Area” slider

The „Area” slider allows users to adjust the size of the measurement area, for which the air pollution dispersion will be measured. It’s possible to choose the value from 1 km² to 25 km². Increasing the area may cause the computation engine to work longer. The default value is set to 4 km².

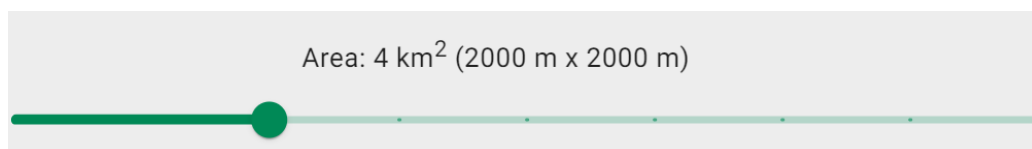


Figure 1.17. „Area” slider with default value of 4 km² (2000 m x 2000 m) chosen.

After choosing the appropriate value, the area on which measurements will be carried out will be visualized on the map in the form of blue square.

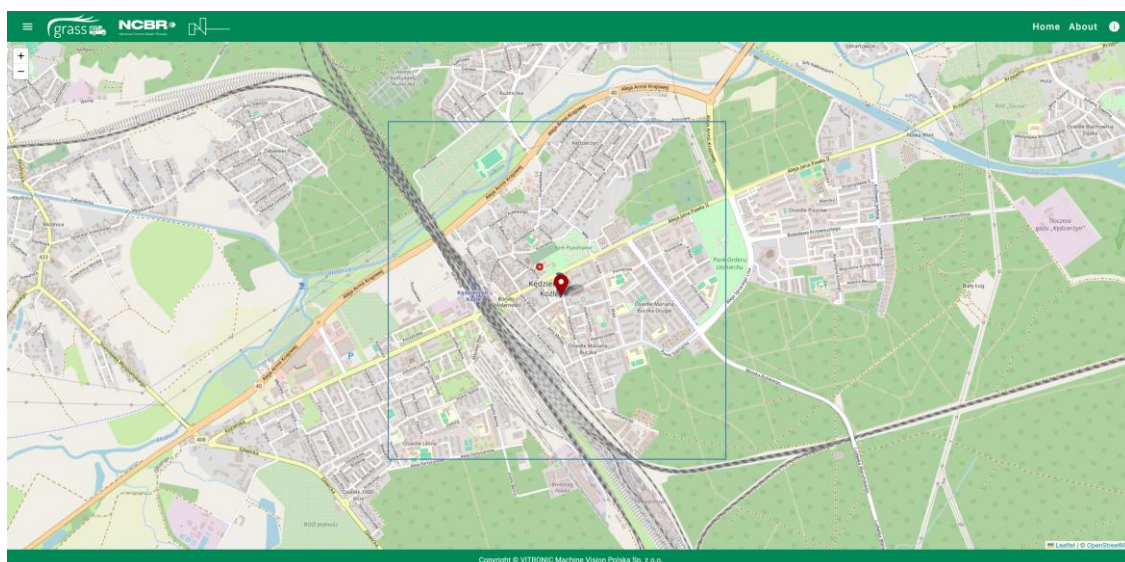


Figure 1.18. Blue square representing the measurement area.

1.3.9 “Calculate” button

After all form fields are filled, the “Calculate” button will be automatically unlocked. By clicking it the HTTP request containing all necessary data will be sent to the server. Then the measurement with the specified parameters will be carried out. The calculation time is highly dependent on values set by user during the configuration of parameters.

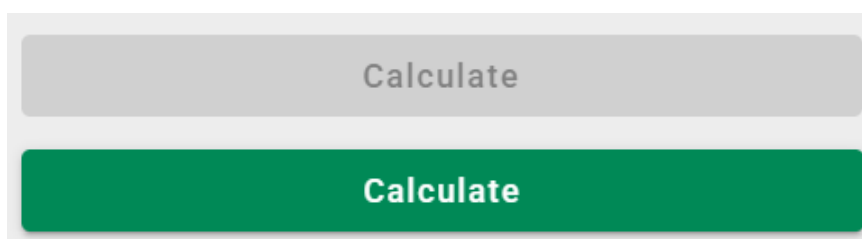


Figure 1.19. Inactive and active „Calculate” button.

1.3.10 “Clear” button

The „Clear” button allows users to clear all filled form fields.

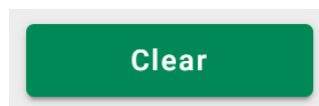


Figure 1.20. „Clear” button.

1.3.11 Loading screen

After the request is sent to the server, the application will wait for the response from the computational engine. During this time user can't interact with neither the form nor the map. After the response from mathematical model is sent back, the overlay will be automatically closed.

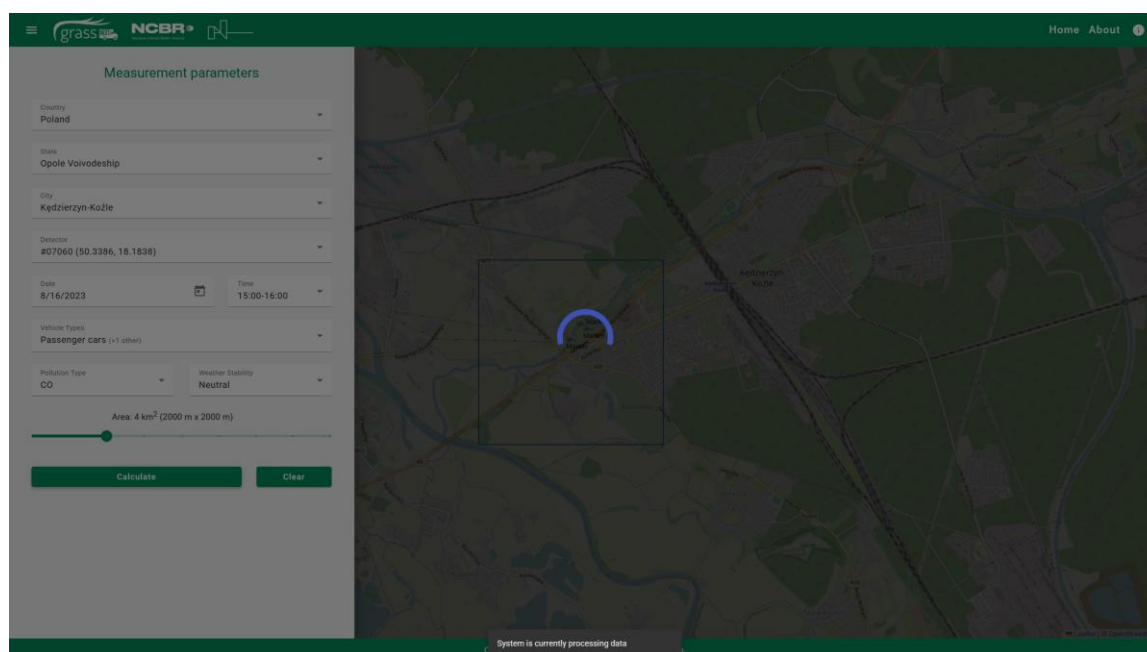


Figure 1.21. Inactive and active „Calculate” button.

1.3.12 Additional information

Each time additional information must be displayed, it appears in form of a notification at the bottom of the site. It can be manually closed by clicking the “Close” button or it'll be automatically removed from screen after the set amount of time.

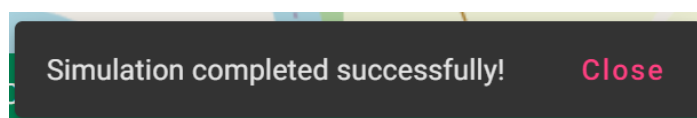


Figure 1.22. Notification window.

1.4 Map

After filling the form all data will be send to the mathematical model for computation. Measurement data received during the calculation process will be displayed on the map in form of isolines, which color is dependent on the chosen pollution type. Additionally, after dragging the mouse cursor over the selected isoline, it'll show the air pollution value for the chosen threshold in $\mu\text{g}/\text{m}^3$.

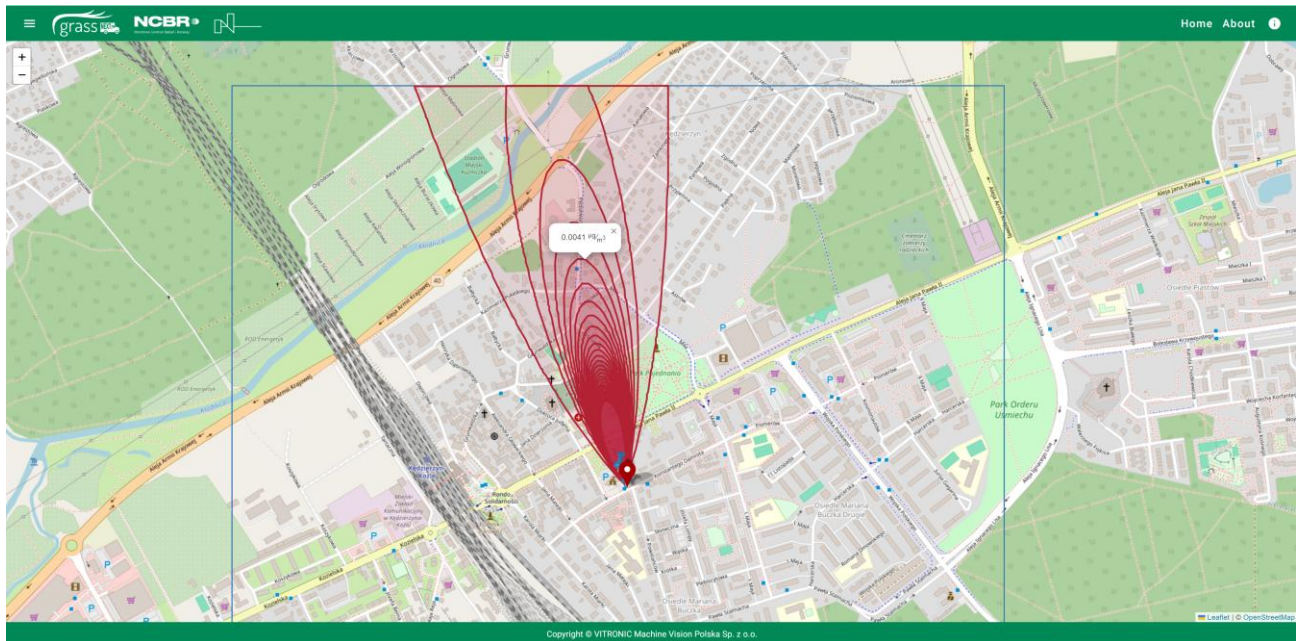


Figure 1.23. Map with marked isolines.

1.5 Footer

Information about the company that was responsible for creation of the application (Vitronic), along with the link to its appropriate homepage, is contained in the footer at the bottom of the website.

Copyright © VITRONIC Machine Vision Polska Sp. z o.o.

Figure 1.24. Footer with company name.

1.6 “About” subsite

The second of the available subsites contains information about GRASS-NEXT project, as well as logos of affiliated partners (NCBR, Norway Grants, Faculty of Economics and Transport Engineering of Maritime University of Szczecin, Institute of Transport Economics and Vitronic). Each logo contains link, which redirects user to the site of appropriate organization.

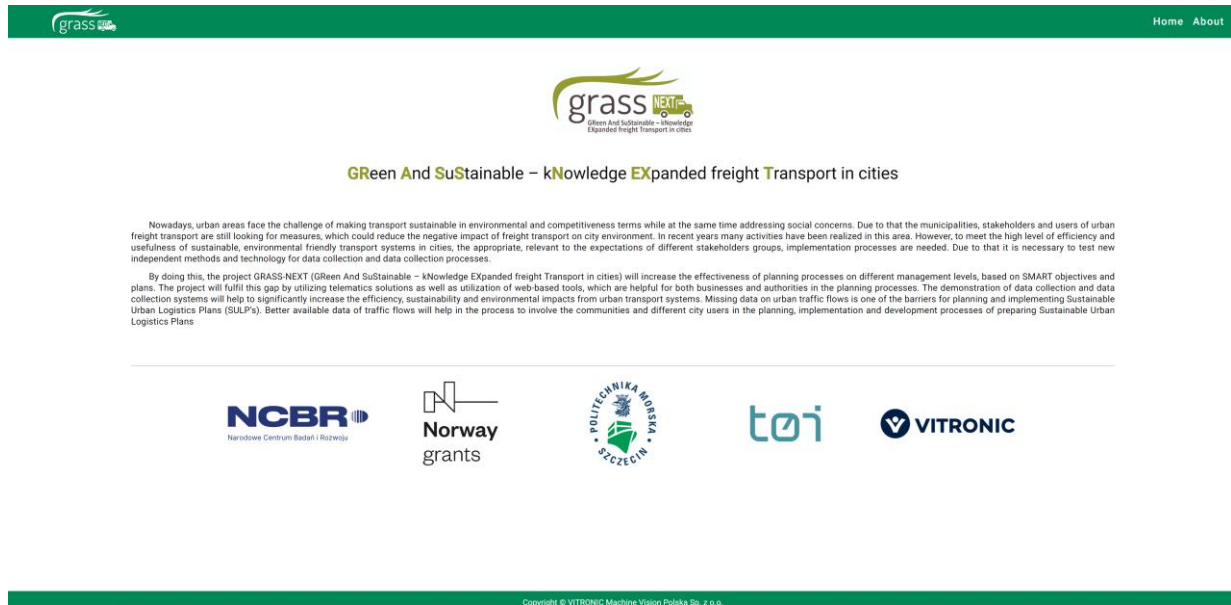


Figure 1.25. „About” subsite.

2 TECHNICAL DOCUMENTATION

2.1 Documentation purpose

The purpose of this document is to present each step of design, creation and implementation of the www.green-tronic.eu website.

The following documentation will contain detailed description of how each app component operates, as well as the technologies necessary to execute this project. Additionally, the use cases with chosen diagrams will be presented.

2.2 Design and implementation constraints

Service will be available on the www.green-tronic.eu website through the browser on either PC or mobile device, as well as through the related [GitHub repository](#). The system needs work twenty-four hours a day, which causes that an employee should monitor it. Part of the application is based on availability of the third-party API's used to gather necessary data, which requires the control of its accessibility and correctness of data received.

The system also can't exceed the upper threshold for RAM and disk space consumption due to the limitation of the purchased server. It's also necessary to adjust the computation time of mathematical model through optimization to match the power of the device on which this component is run.

2.3 Functional requirements

In the following part of the documentation functional requirements of the application will be presented and described in detail.

2.3.1 List of functional requirements

In this section of the documentation list of functional requirements will be presented.

ID	Requirements
01	Application structure
02	TOPO files manager

Table 2.1. Functional requirements.

2.3.2 Detailed description of functional requirements

1. **Use case identifier:** 01
2. **Name:** Application structure
3. **Business justification:** User need to interact with website to input correct measure parameters in order for system to calculate air pollution dispersion.
4. **Actors:** user, database, mathematical model
5. **Scenarios:**
 - Main scenario:
 - **Starting requirements:** User must visit the website.
 - **Course of action:**

1. User fills all the fields of the form (country, state, city, detector, date, time, vehicle type, pollution type, weather stability) and adjusts the area slider.
 2. User presses the “Calculate” button.
 3. A request is sent to the server.
 4. The server communicates with database to retrieve necessary data.
 5. The server sends necessary data to the mathematical model.
 6. The mathematical model sends results back to the server.
 7. Server forwards the data to the frontend.
 8. Isolines are displayed on the map.
- **Effects:** Isolines representing air pollution dispersion for set parameters are displayed on map.
 - **Non-functional requirements:** User should see the results of the calculation in the form of isolines presented on map within one minute of sending a request to the server.
 - **Frequency:** 5
 - **Importance:** 5
- Alternative scenario No. 1:
 - **Starting requirements:** User must visit the website. Data sent to frontend was incomplete or the sample was too small.
 - **Course of action:**
 1. User fills all the fields of the form (country, state, city, detector, date, time, vehicle type, pollution type, weather stability) and adjusts the area slider.
 2. User presses the “Calculate” button.
 3. A request is sent to the server.
 4. The server communicates with database to retrieve necessary data.
 5. The server sends necessary data to the mathematical model.
 6. The mathematical model sends results back to the server.
 7. Server forwards the data to the frontend.
 8. An information about an error is displayed on the screen.
 - **Effects:** A notification is presented in the bottom of the screen in the form of a notification that informs user about the issue.
 - **Non-functional requirements:** The notification will be displayed for 5 seconds with an appropriate information for the user.
 - **Frequency:** 2
 - **Importance:** 3

- Alternative scenario No. 2:
 - **Starting requirements:** User must visit the website. Calculation failed due to incomplete data.
 - **Course of action:**
 1. User fills all the fields of the form (country, state, city, detector, date, time, vehicle type, pollution type, weather stability) and adjusts the area slider.
 2. User presses the “Calculate” button.
 3. A request is sent to the server.
 4. The server communicates with database to retrieve necessary data.
 5. The server sends necessary data to the mathematical model.
 6. The mathematical model sends the information about an error back to the server.
 7. Server forwards the information about an error to the frontend.
 8. An information about an error is displayed on the screen.
 - **Effects:** A notification is presented in the bottom of the screen in the form of a notification that informs user about the issue.
 - **Non-functional requirements:** The notification will be displayed for 5 seconds with an appropriate information for the user.
 - **Frequency:** 1
 - **Importance:** 3

- Alternative scenario No. 3:
 - **Starting requirements:** User must visit the website. Database failed to retrieve necessary data.
 - **Course of action:**
 1. User fills all the fields of the form (country, state, city, detector, date, time, vehicle type, pollution type, weather stability) and adjusts the area slider.
 2. User presses the “Calculate” button.
 3. A request is sent to the server.
 4. The server can’t communicate with database to retrieve necessary data.
 5. Server forwards the information about an error to the frontend.
 6. An information about an error is displayed on the screen.
 - **Effects:** A notification is presented in the bottom of the screen in the form of a notification that informs user about the issue.
 - **Non-functional requirements:** The notification will be displayed for 5 seconds with an appropriate information for the user.
 - **Frequency:** 2
 - **Importance:** 3

- Alternative scenario No. 4:
 - **Starting requirements:** User must visit the website. An error occurred when sending data to the server (e.g. Internet connection issues).
 - **Course of action:**
 1. User fills all the fields of the form (country, state, city, detector, date, time, vehicle type, pollution type, weather stability) and adjusts the area slider.
 2. User presses the “Calculate” button.
 3. A request can’t be sent to the server due to an error.
 4. An information about an error is displayed on the screen.
 - **Effects:** A notification is presented in the bottom of the screen in the form of a notification that informs user about the issue.
 - **Non-functional requirements:** The notification will be displayed for 5 seconds with an appropriate information for the user.
 - **Frequency:** 3
 - **Importance:** 3
 -
- Alternative scenario No. 5:
 - **Starting requirements:** User must visit the website. User pressed the “Clear” button.
 - **Course of action:**
 1. User fills all the fields of the form (country, state, city, detector, date, time, vehicle type, pollution type, weather stability) and adjusts the area slider.
 2. User presses the “Clear” button.
 3. All of the filled form fields are cleared.
 - **Effects:** All of the fields of the form that have been filled by the user are cleared. The default value of “None” is set.
 - **Non-functional requirements:** The values will be cleared immediately after pressing the “Clear” button. User can’t press the “Calculate” button, so the invalid request can’t be sent to the server.
 - **Frequency:** 3
 - **Importance:** 4
- Alternative scenario No. 6:
 - **Starting requirements:** User must visit the website. User changed the value of any field of the form.
 - **Course of action:**
 1. User fills all the fields of the form (country, state, city, detector, date, time, vehicle type, pollution type, weather stability) and adjusts the area slider.
 2. User changed the value of any of the form fields’.
 3. All of the following field form fields will change value to the default one (“None”).

- **Effects:** All of the following form fields were cleared. The “Calculate” button did not activate, so it cannot be pressed by the user to calculate air pollution dispersion.
 - **Non-functional requirements:** User can’t press the “Calculate” button, so the invalid request can’t be sent to the server.
 - **Frequency:** 4
 - **Importance:** 5
- Alternative scenario No. 7:
 - **Starting requirements:** User must visit the website. User did not fill all the fields of the form.
 - **Course of action:**
 1. User did not fill all the fields of the form (country, state, city, detector, date, time, vehicle type, pollution type, weather stability) and adjusts the area slider.
 2. The “Calculate” button is still inactive.
 - **Effects:** The “Calculate” button did not activate, so it cannot be pressed by the user to calculate the air pollution dispersion.
 - **Non-functional requirements:** User can’t press the “Calculate” button, so the invalid request can’t be sent to the server.
 - **Frequency:** 5
 - **Importance:** 5

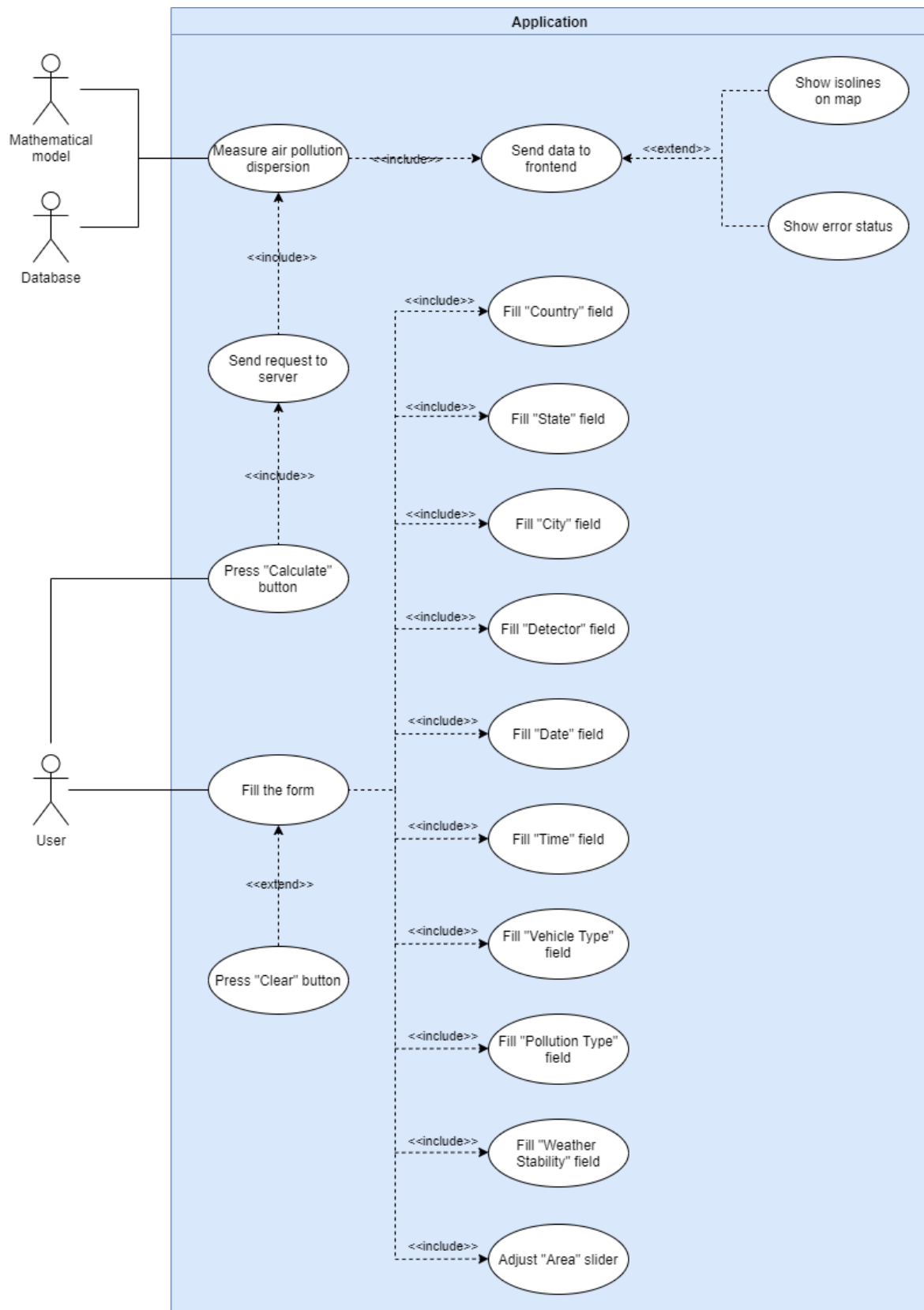


Figure 1.1. Use case No. 1: Application structure.

1. **Use case identifier:** 02
2. **Name:** TOPO files manager
3. **Business justification:** The file watcher that works twenty-four hours a day is needed to retrieve information from TOPO detectors, which send files to a designated folder on the server. TOPO files need to be parsed properly and all the necessary data needs to be saved to the database in order to properly calculate air pollution dispersion.
4. **Actors:** TOPO files, Nominatim API, Overpass Turbo API, OpenMeteo API, database
5. **Scenarios:**
 - Main scenario:
 - **Starting requirements:** New TOPO file appeared in an designated folder on the server. The TOPO detector didn't already exist in the database with relation to its current location.
 - **Course of action:**
 1. New TOPO file is read by the server.
 2. The data from TOPO file is parsed and split into multiple elements.
 3. The data about TOPO detector and its location is received from Nominatim and Overpass Turbo APIs.
 4. New TOPO detector is created as an object.
 5. The data from TOPO file is aggregated hourly.
 6. Information about weather for the day the TOPO detector gathered data is received from OpenMeteo API.
 7. Weather, TOPO detector and aggregated data are saved in the database.
 - **Effects:** All the data received form the new TOPO file was parsed and saved to the database. New detector was successfully added.
 - **Non-functional requirements:** All operation are processed in a transaction to ensure integrity of the database.
 - **Frequency:** 4
 - **Importance:** 5
 - Alternative scenario No. 1:
 - **Starting requirements:** New TOPO file appeared in an designated folder on the server. The TOPO detector did already exist in the database with relation to its current location.
 - **Course of action:**
 1. New TOPO file is read by the server.
 2. The data from TOPO file is parsed and split into multiple elements.
 3. The data from TOPO file is aggregated hourly.
 4. Information about weather for the day the TOPO detector gathered data is received from OpenMeteo API.
 5. Weather, TOPO detector and its aggregated data are saved in the database.
 - **Effects:** All the data received form the new TOPO file was parsed, aggregated and saved to the database.

- **Non-functional requirements:** All operation are processed in a transaction to ensure integrity of the database.
- **Frequency:** 4
- **Importance:** 5
- Alternative scenario No. 2:
 - **Starting requirements:** New TOPO file appeared in an designated folder on the server. Data could not be saved in the database.
 - **Course of action:**
 1. New TOPO file is read by the server.
 2. The data from TOPO file is parsed and split into multiple elements.
 3. The data from TOPO file is aggregated hourly.
 4. Information about weather for the day the TOPO detector gathered data is received from OpenMeteo API.
 5. Weather, TOPO detector and aggregated data could not be saved in the database.
 - **Effects:** All the data received form the new TOPO file was parsed and aggregated, but couldn't be saved to the database.
 - **Non-functional requirements:** All operation are processed in a transaction to ensure integrity of the database.
 - **Frequency:** 3
 - **Importance:** 5
- Alternative scenario No. 3:
 - **Starting requirements:** New TOPO file appeared in an designated folder on the server. The TOPO detector did already exist in the database with relation to its current location. Weather could not be received from the OpenMeteo API.
 - **Course of action:**
 1. New TOPO file is read by the server.
 2. The data from TOPO file is parsed and split into multiple elements.
 3. The data from TOPO file is aggregated hourly.
 4. Information about weather for the day the TOPO detector gathered data is received from OpenMeteo API.
 5. Weather, TOPO detector and aggregated data are saved in the database.
 - **Effects:** All the data received form the new TOPO file was parsed and aggregated, but couldn't be saved to the database.
 - **Non-functional requirements:** All operation are processed in a transaction to ensure integrity of the database.
 - **Frequency:** 3
 - **Importance:** 5
- Alternative scenario No. 4:

- **Starting requirements:** New TOPO file appeared in an designated folder on the server. The TOPO detector did already exist in the database with relation to its current location. Data could not be properly aggregated.
 - **Course of action:**
 1. New TOPO file is read by the server.
 2. The data from TOPO file is parsed and split into multiple elements.
 3. The data about TOPO detector and its location is received from Nominatim and Overpass Turbo APIs.
 4. The data from TOPO file could not be aggregated hourly due to an error.
 - **Effects:** All the data received form the new TOPO file was parsed, but couldn't be aggregated or saved to the database.
 - **Non-functional requirements:** All operation are processed in a transaction to ensure integrity of the database.
 - **Frequency:** 1
 - **Importance:** 5
- Alternative scenario No. 5:
 - **Starting requirements:** New TOPO file appeared in an designated folder on the server. The TOPO detector did already exist in the database with relation to its current location. Data could not be properly parsed.
 - **Course of action:**
 1. New TOPO file is read by the server.
 2. The data from TOPO file could not be parsed properly due to an error.
 - **Effects:** All the data received form the new TOPO couldn't be parsed, aggregated or saved to the database.
 - **Non-functional requirements:** All operation are processed in a transaction to ensure integrity of the database.
 - **Frequency:** 1
 - **Importance:** 5
- Alternative scenario No. 6:
 - **Starting requirements:** New TOPO file appeared in an designated folder on the server. The TOPO detector did already exist in the database with relation to its current location. Data could not be properly read from TOPO file.
 - **Course of action:**
 1. New TOPO file could not be read by the server.
 - **Effects:** All the data received form the new TOPO couldn't be parsed, aggregated or saved to the database.
 - **Non-functional requirements:** All operation are processed in a transaction to ensure integrity of the database.
 - **Frequency:** 1
 - **Importance:** 5

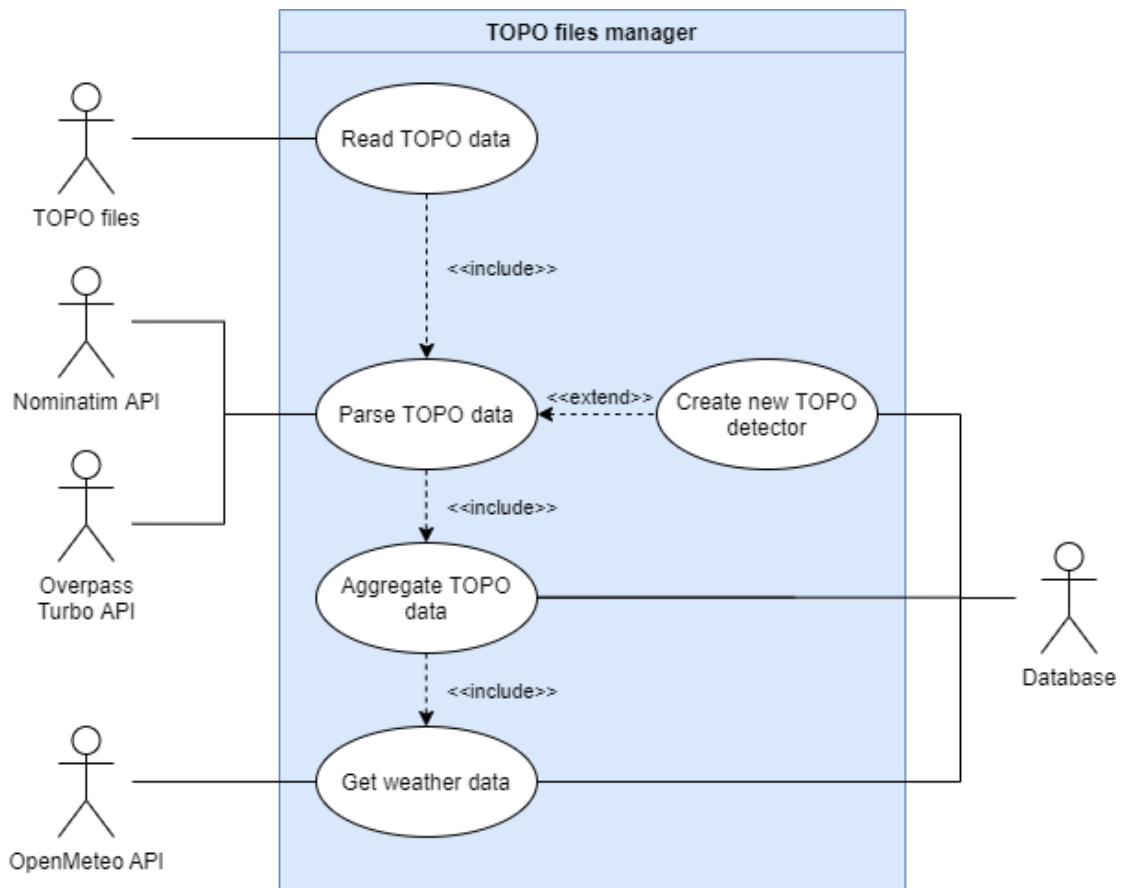


Figure 1.2. Use case No. 2: TOPO files manager.

2.4 Non-functional requirements

Below all of the non-functional requirements that are applicable to the application presented in this documentation will be described in detail:

1. **Efficiency** – the calculation of the mathematical model are processed in time dependent on the size of the area, namely from 5 to 60 seconds (assuming minimal use of RAM). It's also worth noting that the time of communication between frontend and backend server were optimized through reduction of redundant data sent through the HTTP request (only information about point forming isolines is delivered and not the whole matrix).
2. **Security** – application doesn't have such elements as user registration, which means it doesn't require security capabilities in the aspect of preservation of client data. System is based mostly on protection of information stored in database and avoidance of malicious third-party attacks such as SQL injections or XSS (Cross-site scripting).
3. **Scalability** – application was designed in such a way that future changes such as increase of the area, for which the air pollution dispersion is calculated by

mathematical model is easy to introduce without the need of fundamental changes in the system.

4. **Availability** – application should be available twenty-four hours a day through the www.green-tronic.eu website. In case any critical error occurs, it's anticipated that the service window will exist until the problem is solved.
5. **Conservation** – thanks to the existence of application backups it's possible to recreate whole site to its previous state after critical error occurs. In this case it's necessary to also load all TOPO files to the database through designated folder on the server.
6. **Usability** – application enables users to find information about air pollution dispersion. It's an element rarely encountered on sites with similar topics and themes, which makes the existence of aforementioned system unique.
7. **Reliability** – application should generate as few errors as possible. Due to the nature of the work, it would be advisable to focus on securing data collected from TOPO detectors by creating regular backup copies. Eventual errors, which would in turn cause unavailability of the system, assuming quick reaction from site administrators, shouldn't hinder users to a large degree from using the website.
8. **Verification** – system is consistent with all established norms, such as using generally accepted coordinate WGS 84 standard, occurrence of logos of affiliated partners as well as using free API's to gather necessary data, which means no additional costs are needed except that of maintaining server on which the application and database are located.

2.5 System architecture

Internet website of GRASS-NEXT project allows users to analyse the air pollution dispersion. After inputting necessary data, based on the results received from the mathematical model, isolines are presented on the map. They show the concentration levels of selected compound. In the following section a detailed architecture of the implemented system will be presented alongside the description of all used technologies and created components.

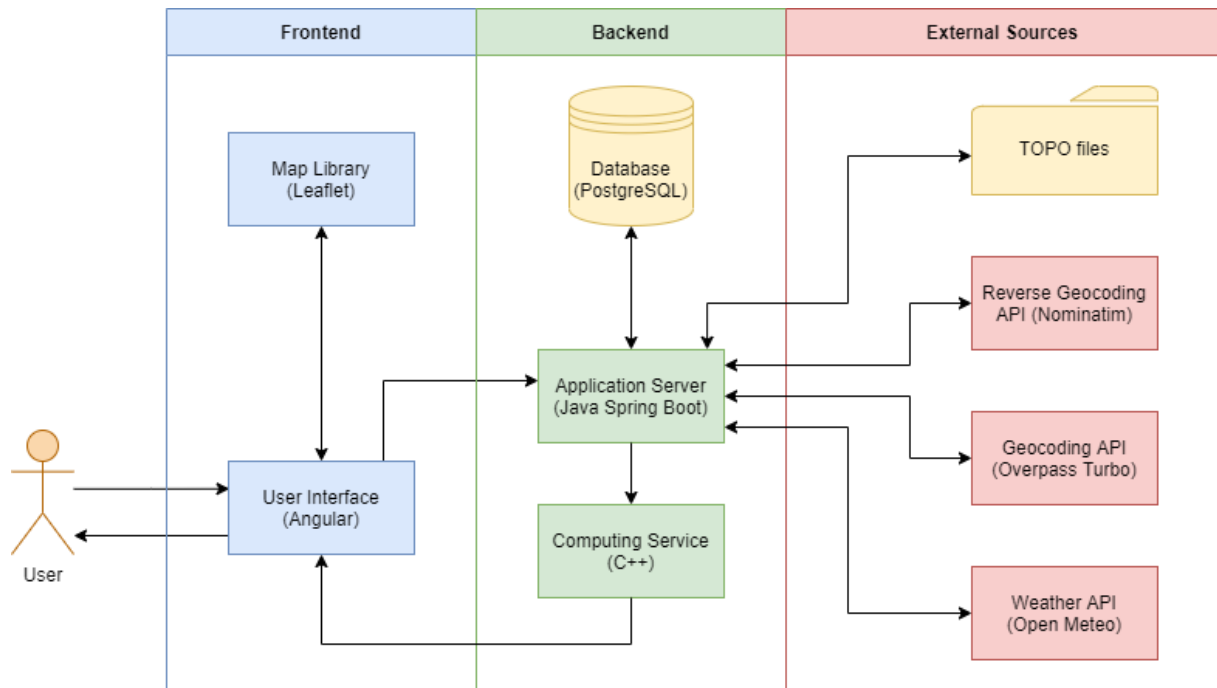


Figure 2.3. The three-layer architecture of a computer program for modelling the pollution level from transport.

2.5.1 Technologies used

For the implementation of the application the following technologies were used to create necessary components:

- website was created using Angular framework with Typescript,
- application server was implemented using Java Spring Boot,
- database was created using PostgreSQL,
- mathematical model is implemented in C++ as an additional library.

To find all the necessary information about the placement of road segment (its beginning and end), as well as about location (country, state, city) based on geographic coordinates read from detector, two API calls are made. The services used in this case are called Overpass Turbo (service based on OpenStreetMap that gathers information about road segments) and Nominatim (service for reverse geocoding allowing user to get information about locations based on coordinates). Necessary weather data (wind speed and direction) are collected from OpenMeteo API.

ID	Technology	Explanation
01	Java with Spring Boot framework and REST	This technology can be used free of charge and doesn't increase costs of using the system by customer. All data components are shared through REST technology through appropriate controllers.

02	PostgreSQL	This technology can be used free of charge and doesn't increase costs of using the system by customer. It also meets all standards necessary to securely store data inside database created for the GRASS-NEXT project.
03	HTML5, Typescript with Angular Framework, CSS3 with Angular Material library for styling	This technology can be used free of charge and doesn't increase costs of using the system by customer. Designing websites using Angular library and Angular Material styles is intuitive, which will allow for faster creation of user interface. Application will send HTTP request to Java Spring Boot server. Necessary data will be in turn received through responses sent by backend REST controllers.
04	C++	This technology can be used free of charge and doesn't increase costs of using the system by customer. C++ allows for very fast computation of large matrices. It's necessary to perform complicated data analysis and calculations of air pollution dispersion.

Table 2.2. Technologies used.

2.5.2 Communication between frontend and backend

The system is based on communication between frontend side (Angular framework) and computational server (Java Spring Boot). User is presented with the map displayed with help of Leaflet library. After filling the form with measurement parameters, website sends an HTTP request to server, which in turn remodels the data structure in such a way that it's possible for mathematical model to analyse incoming information.

Data about air pollution type and vehicle types, chosen TOPO detector, as well as congruent weather stability, wind direction and speed for selected date and time, are prepared.

New TOPO sensors are added to the database daily about noon after being analysed by server.

2.5.3 Mathematical model

After preparing the data, JSON request is sent to the mathematical model implemented as an C++ library. It analyses incoming information and calculates appropriate coefficients for the corresponding fields of air pollution matrix. Points of such structure are 1 meter away from each other. They are later analysed by opencv2 library to create isolines based on set thresholds, which are in turn sent in a request back to the server and later to the website. Next they are visible to user after being drawn on a map.

2.5.4 Database design

For this application a database was created using PostgreSQL technology. It's necessary to store historical data, which in turn allows user to measure air pollution dispersion for past dates.

The main purpose of such database is the storage of static, unchangeable information such as weather stability, pollution or vehicle types, as well as data about vehicle count and average velocity for each group. The TOPO sensors themselves with their location are also saved there. The last important element stored is information about weather, such as wind speed and wind direction. Without it, it would be impossible to recreate measurements for past dates.

All stored data are aggregated hourly, so not each vehicle is recorder. It allows for the optimization of both API request and reducing disk space occupied by the database.

Below all tables created for this project are described in detail:

- **Weather Stability** – static table containing data about available weather stability types (extremely unstable, moderately unstable, slightly unstable, neutral, slightly stable, moderately stable),
- **Vehicle Group** – static table containing data about available vehicle groups (motorcycles, passenger cars, light commercial vehicles, heavy goods vehicles, public transport buses) ,
- **Pollution Type** – static table containing data about available pollution types (carbon monoxide, nitrogen oxides, hydrocarbons),
- **Topo Detector** – table containing data about TOPO detectors, which data was gathered and sent to the server. Sensors are described by their identifier and geographic coordinates. Each record in the database is connected to the specified location,
- **Location Data** – table containing data about locations, in which TOPO detectors were placed. It contains information about country, state, city and road, alongside which the sensor was placed (via Nominatim API), as well as its beginning and end of such road segment (via Overpass Turbo API),
- **Topo Aggregated Data** – table containing aggregated data collected from TOPO detectors. Information about vehicle count and their average velocity are previously averaged into hourly groups. Each record is connected to the identifier of the sensor that gathered respective data.
- **Weather** – table containing information about weather conditions (wind speed and direction) for each hour of any of the recorded days. It corresponds to aggregated TOPO data.

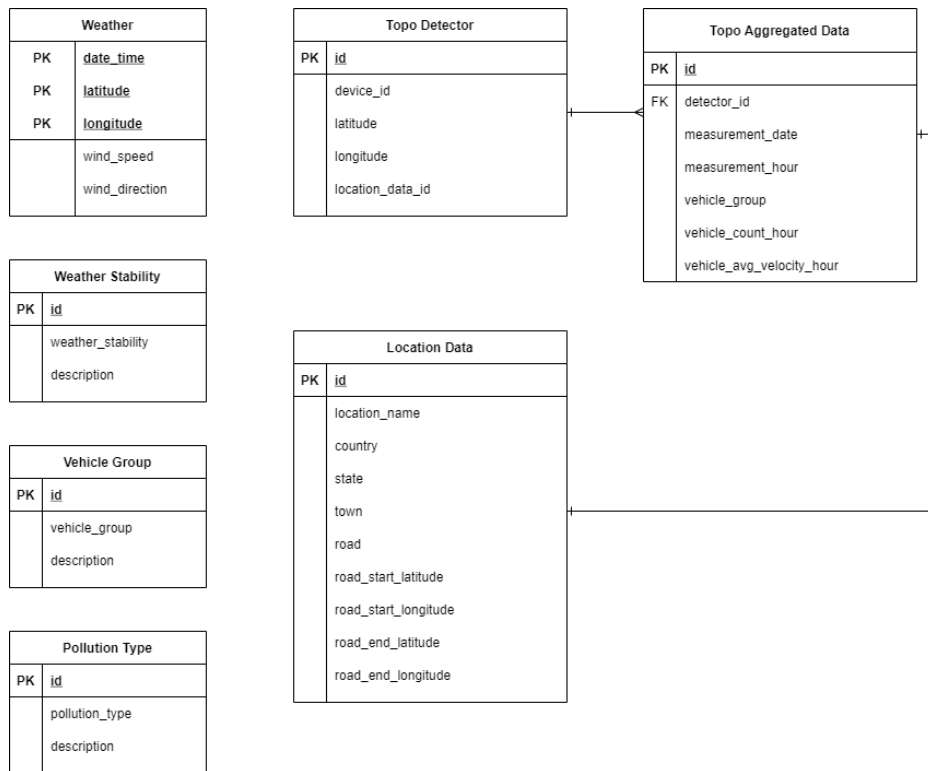


Figure 2.4. The scheme of a database for storing data from sensors placed on roads..

2.6 Test scenarios

In this section test scenarios for the application described in this documentation will be presented in the form of a table.

ID	Test scenarios
01	Testing the correctness of “Menu” icon (hiding and showing form)
02	Testing the correctness of the links embedded into affiliated partners’ logs (redirecting users to their homepages)
03	Testing the correctness of “Home” and “About” buttons (redirecting to the correct subpage)
04	Testing the correctness of the “Info” icon (displaying legend)
05	Testing the correctness of the link embedded into the footer (redirecting to Vitronic homepage)
06	Testing the correctness of the “Country” form field (sending requests to the server and receiving list of available countries)
07	Testing the correctness of the “State” form field (sending requests to the server and receiving list of available states for the given country)
08	Testing the correctness of the “City” form field (sending requests to the server and receiving list of available cities for the given state)
09	Testing the correctness of the “Detector” form field (sending requests to the server and receiving list of available detectors for the given location)
10	Testing the correctness of the “Date” form field (sending requests to the server and receiving list of all the dates, in which the detector gathered data)

11	Testing the correctness of the “Time” form field (displaying all available one hour intervals)
12	Testing the correctness of the “Vehicle Type” form field (sending requests to the server and receiving list of available vehicle types)
13	Testing the correctness of the “Pollution Type” form field (sending requests to the server and receiving list of available pollution types)
14	Testing the correctness of the “Weather Stability” form field (sending requests to the server and receiving list of available weather stabilities)
15	Testing the correctness of the “Area” slider (adjusting the size of measured area)
16	Testing the correctness of the “Clear” button (clearing all of the form fields)
17	Testing the correctness of the “Measurement” button (sending correct requests to the servers)
18	Testing the correctness of the links embedded into logos on the “About” page (redirecting to the homepages of respective partners)
19	Testing the correctness of the map displaying in the UI
20	Testing the correctness of isolines displaying on the map after receiving data from the mathematical model
21	Testing the correctness of the server endpoint (correctly receiving and parsing data)
22	Testing the correctness of the file watcher (waiting for TOPO files to appear and parsing their content after they are send to the server)
23	Testing the correctness of the database (correctly processing transactional queries in order to save necessary data)
24	Testing the correctness of the mathematical model (correctly evaluating incoming matrices to produce results comparable to the ones received from the base model)
25	Testing the correctness of the connection between frontend and backend (correctly sending and receiving responses from the server)
26	Testing the correctness of the backend server and mathematical model (transferring data between server and additional library containing computing engine)
27	Testing the correctness of the APIs used in the process (Nominatim, Overpass Turbo, OpenMeteo)
28	Testing the application on multiple types of devices of different resolution (PC, mobile)

Table 2.3. Technologies used.